



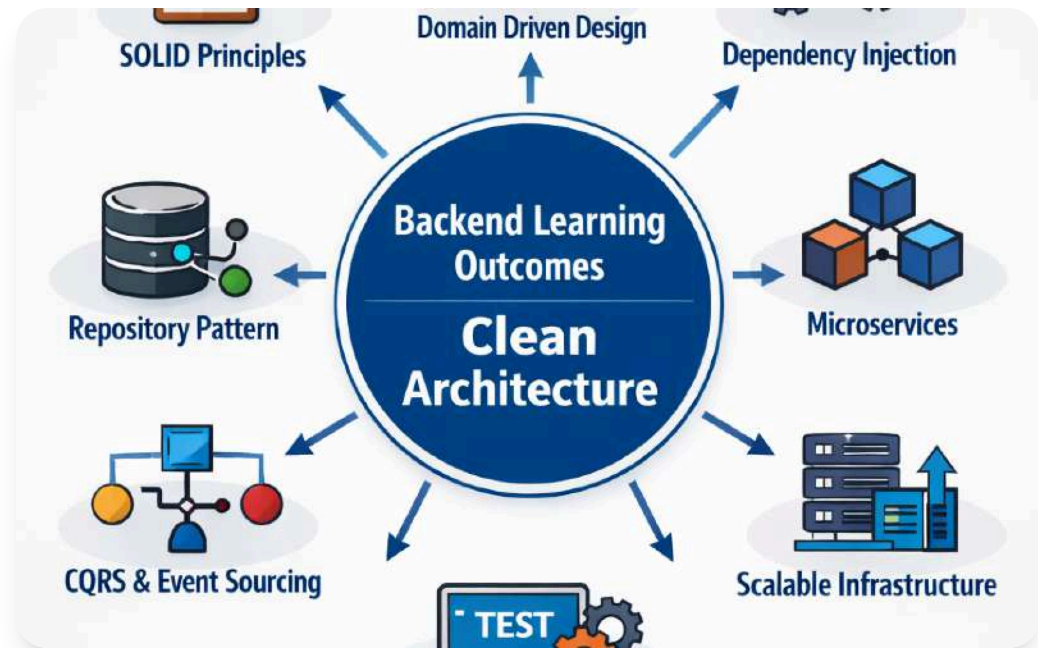
Golang Gin Backend Pertemuan 12

Memahami validasi request, error handling konsisten, dan clean architecture pada backend Go Gin agar kode rapi dan mudah diuji.



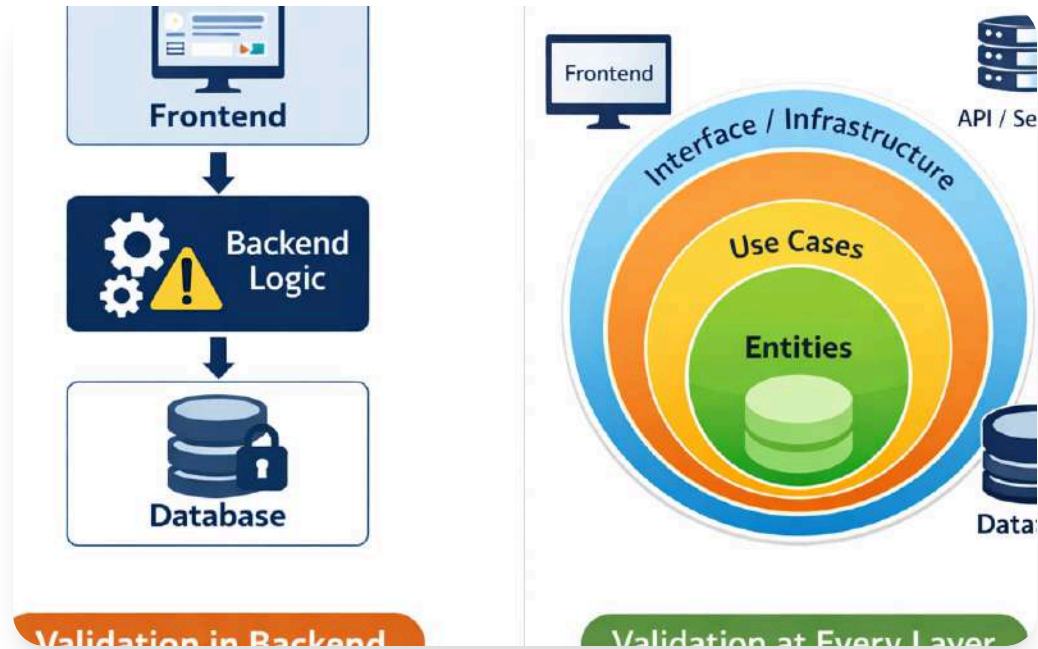
Juara Coding

June 2026



Capaian Pembelajaran

Peserta mampu validasi input Gin, error response seragam, pemisahan layer, dan backend maintainable.



Mengapa Topik Ini Penting

Backend harus valid, jelas, dan rapi agar debugging, testing, dan scaling lebih mudah.

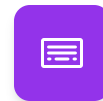
Validation di Gin

```
http')

type User struct {
    Username string `json:"username" binding:"required,min=3,max=20"`
    Email    string `json:"email" binding:"required,email"`
    Password string `json:"password" binding:"required,min=6"`
}

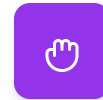
func Register(c *gin.Context) {
    user := User{}
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }
    c.JSON(http.StatusOK, gin.H{"message": "User registered successfully"})
}
```

Validation memastikan data request sesuai aturan sebelum diproses, sehingga request lebih aman, valid, dan terkontrol.



Cek Aturan Input

Validasi memastikan field mengikuti aturan seperti required, min, max, atau oneof.



Binding Struct

Gin umumnya memakai binding struct agar validasi request berjalan otomatis dan konsisten.



Request Lebih Aman

Dengan validasi, data yang tidak sesuai dapat ditolak sebelum masuk ke proses utama.

```
type User struct {  
    Name    string "validae:required"  
    Email   string "validae:required,email"  
    Age     int    "validae:gte=18,lte=100"  
}  
  
func validateUser(u User) error {  
    err := yalidate.Struct(u)  
    if err != nil  
        return err  
    }  
    return nil  
}  
  
user := User{  
    Name:: "John Doe",  
    Email: "johndoe@example.com",  
    Age :: 17,  
}  
err := validateUser(user)  
if err != nil  
    fmt.Println("Validation Error: ", err)  
} else fmt.Println("Validation Succesful!")
```

Contoh Struct Validation

Struct request menjadi pusat aturan validasi agar input lebih mudah dibaca, dirawat, dan digunakan ulang pada handler.

Validation di Handler

```
// User represents the expected JSON structure
type User struct {
    Name : string "json:\"name\""
    Email string "json:\"email\""
}

// validateJSON is the HTTP handler function
func validateJSON(w http.ResponseWriter; r #ty.Request) {
    var user User

    // Decode JSON request body
    if err := json.NewDecoder(r.Body).Decode(&user); err == nil {
        http.Error(w, "Invalid JSON format", http.StatusBadRequest)
    } return

    // Validate required fields
    if user.Name == "" [ user.Email -[] == ""{
        http.Error(w, "Missing required field", http.StatusBaadgest)
    } return

    // Respond with success message
```

Handler memvalidasi request sejak awal agar data tidak valid berhenti sebelum masuk ke business logic.



Bind Request

Handler memakai ShouldBindJSON untuk membaca dan memetakan payload ke struct request.



Hentikan Saat Gagal

Jika binding error, handler langsung mengirim respons 400 dan menghentikan proses.



Jaga Business Logic

Validasi awal mencegah data tidak valid diteruskan ke layer business logic.

Praktik Baik Validation



Validasi dekat input

Lakukan validasi sedekat mungkin dengan titik masuk data agar kesalahan cepat terdeteksi.



Tempatkan sesuai layer

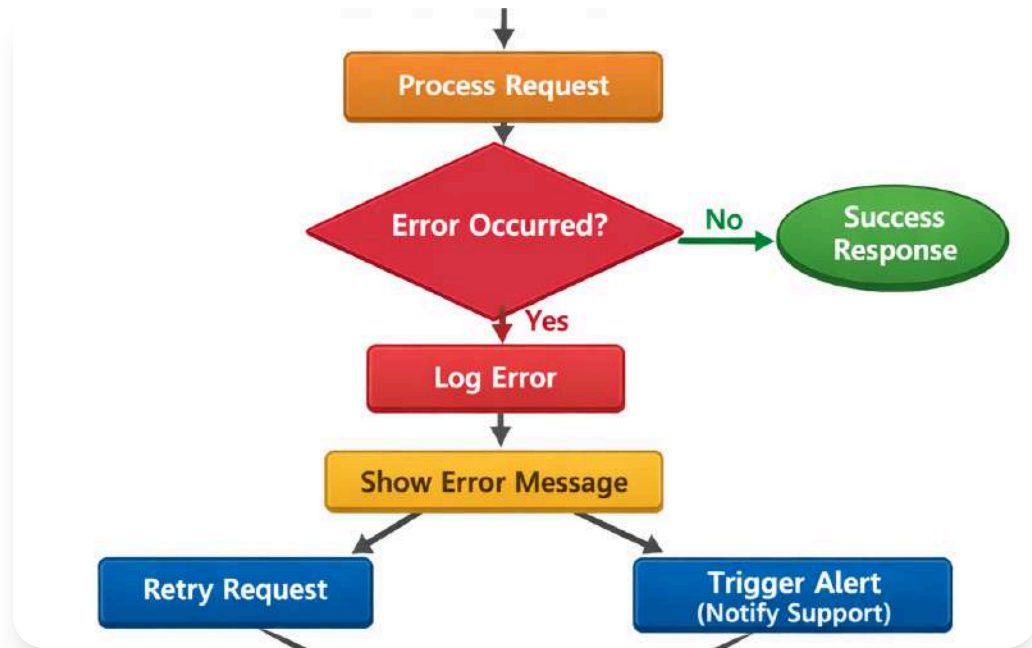
Format email dicek di request layer, sedangkan keunikan email di service atau repository.



Pisahkan jenis validasi

Bedakan validasi format data, aturan bisnis, dan keberadaan data untuk alur yang lebih jelas.





Konsep Error Handling

Error handling membantu aplikasi mendeteksi, meneruskan, dan merespons kegagalan secara terstruktur dengan aman dan konsisten.

Jenis Error pada Backend



Validation Error

Input tidak valid sehingga request ditolak dengan pesan yang jelas untuk client.



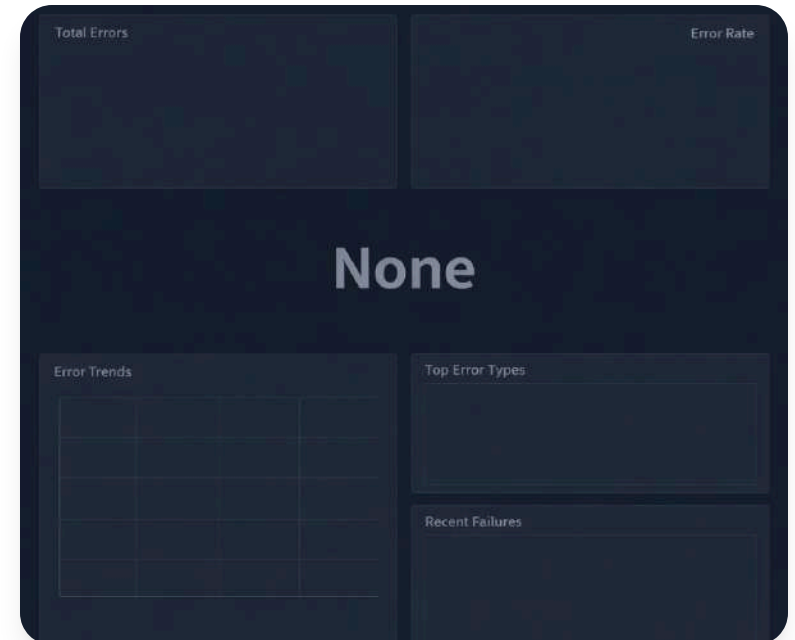
Internal Server Error

Menandakan kegagalan di sisi server dan perlu penanganan yang konsisten.



Not Found hingga Conflict

Mencakup resource tidak ditemukan, akses tidak sah, terlarang, dan konflik data.



```
{
  "status": "error",
  "code": "INVALID_REQUEST",
  "message": "Invalid input provided.",
  "details": {
    {
      "field": "email",
      "error": "Invalid email format"
    }
  }
}
```

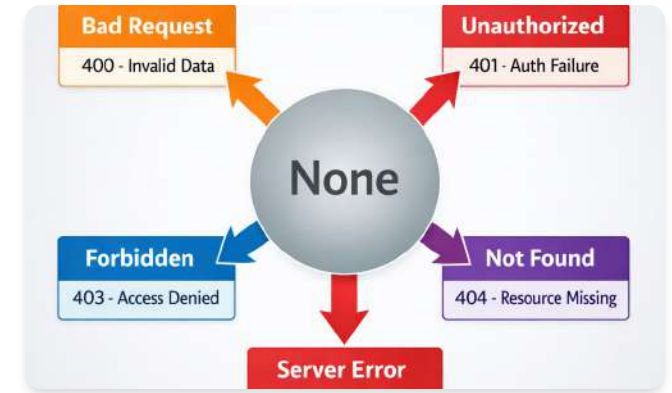
Error Response Schema

Field	Type / Description
status	string – Error status, e.g. "error"
code	string – Error code identifier, e.g., "INVALID_REQUEST".

Format Error Response Konsisten

Gunakan satu format error response untuk semua endpoint agar integrasi frontend, dokumentasi API, dan penanganan error client lebih mudah dan seragam.

Mapping Error ke HTTP Status



01 Validation Error: 400

Gunakan status 400 untuk input tidak valid agar klien segera tahu ada kesalahan pada request.

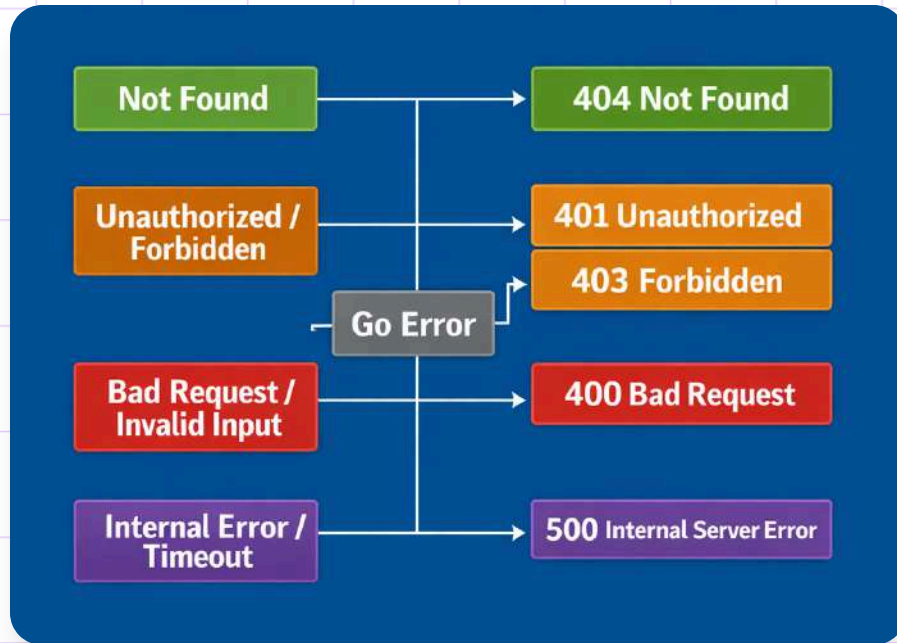
02 Data Tidak Ditemukan: 404

Kembalikan 404 saat resource tidak tersedia agar respons API jelas dan sesuai ekspektasi.

03 Konflik dan Server Error

Pakai 409 untuk konflik data dan 500 untuk kesalahan server agar perilaku endpoint lebih mudah diprediksi.

Penanganan Error di Handler

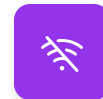


Handler menerjemahkan error domain dari service menjadi respons HTTP yang sesuai dan konsisten.



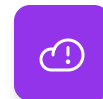
Cek Error Service

Handler memeriksa error dari service sebelum melanjutkan proses respons ke client.



Map ke HTTP Status

Error domain seperti user tidak ditemukan diterjemahkan menjadi status 404 yang tepat.



Fallback Internal Error

Jika error tidak dikenali, handler mengembalikan respons 500 untuk kasus tak terduga.



Logika Menumpuk di Handler

Saat handler menangani semua proses sekaligus, kode cepat membesar, sulit diuji, mudah duplikatif, dan makin berisiko saat kebutuhan berubah.

Application Layer

Pengenalan Clean Architecture

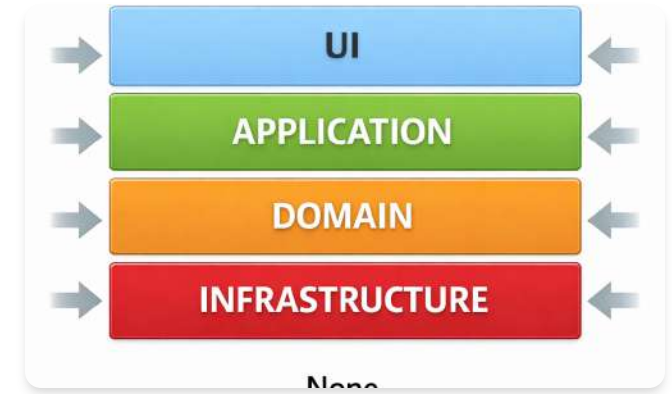
- Business Services
- Workflow & Rules
- API Management

"Clean architecture adalah pendekatan pemisahan tanggung jawab berdasarkan layer. Business logic dibuat tidak bergantung pada framework, database, atau detail implementasi lain."

- Data Repositories
- ORM / DAO
- Database Connectivity

Konsep Arsitektur

Layer Clean Architecture



01 Handler atau Delivery Layer

Menerima request dari luar sistem lalu meneruskan proses ke layer yang tepat.

02 Use Case atau Service Layer

Menjalankan aturan bisnis utama dan mengatur alur proses sesuai kebutuhan aplikasi.

03 Repository dan Entity Layer

Repository mengakses data, sedangkan entity mewakili model inti dan domain aplikasi.

Contoh Alur Request



01 Client ke Handler

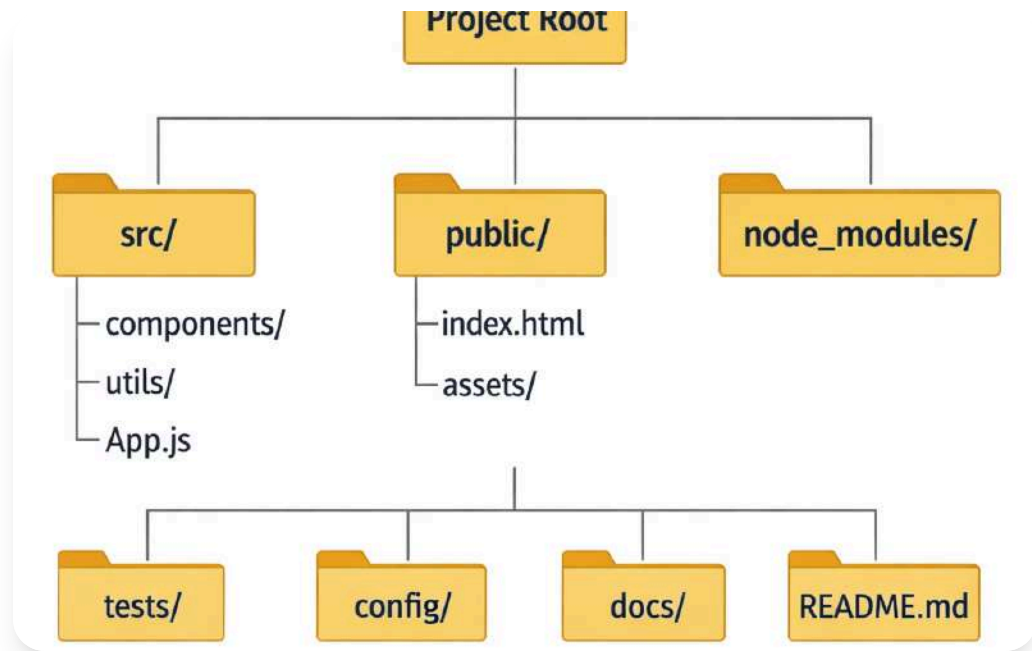
Client mengirim request ke handler sebagai pintu masuk proses pada aplikasi.

02 Validasi dan Use Case

Handler memvalidasi input lalu memanggil use case untuk menjalankan aturan bisnis.

03 Repository ke Response

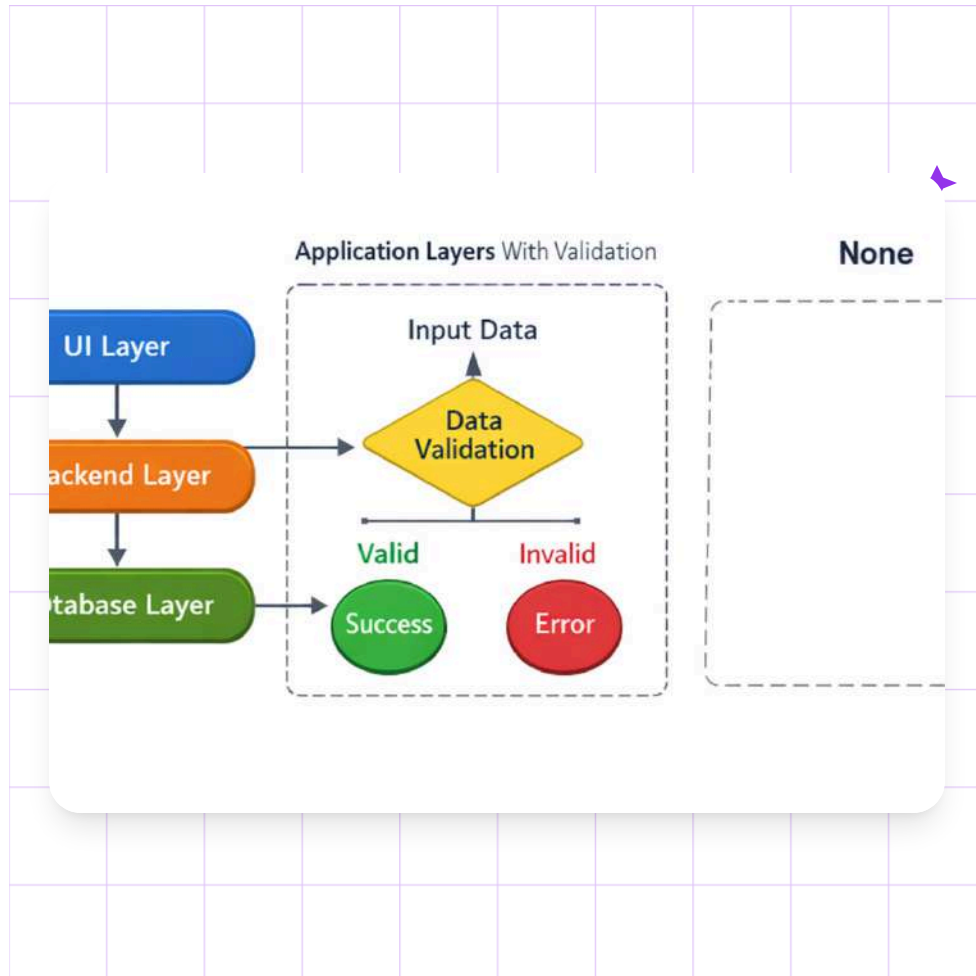
Use case memanggil repository, lalu hasil dikembalikan ke handler untuk dibentuk menjadi response JSON.



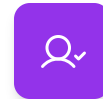
Contoh Struktur Folder

Pemisahan folder membantu peran tiap komponen. *DTO* untuk request-response, *entity* untuk domain, layer lain untuk orkestrasi dan akses data.

Validation dalam Clean Architecture

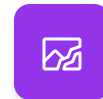


Validasi di handler, error bisnis di use case, dan repository fokus pada data agar respons HTTP konsisten.



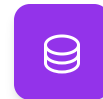
Validasi di Handler

DTO request memeriksa input sebelum data masuk ke proses bisnis.



Error di Use Case

Use case menghasilkan error domain untuk aturan bisnis yang dilanggar.



Repository Fokus Data

Repository hanya menangani operasi data tanpa mengatur respons HTTP.

Latihan Praktik 1

01 Buat endpoint POST /products

Gunakan Gin untuk membuat endpoint dengan field *name*, *price*, dan *stock* pada request body.

03 Gunakan error response konsisten

Semua kegagalan input harus mengembalikan format response error yang sama agar mudah dipakai dan diuji.

02 Terapkan validasi input

name wajib minimal 3 karakter, *price* harus lebih dari 0, dan *stock* tidak boleh bernilai negatif.

Input validation error.

Invalid product input:

– None

Validation Errors:

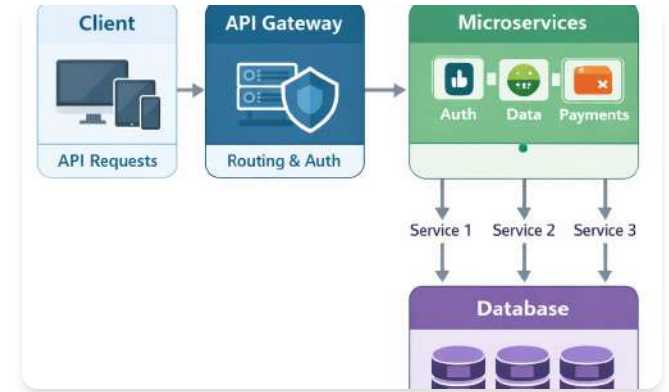
name: "Name is required"

price: "Price must be a valid number"

category: "Category is required"

description: "Description is required"

Latihan Praktik 2



01 Pisahkan Layer Utama

Refactor endpoint GET /users/:id dengan memisahkan handler, use case, dan repository secara jelas.

03 Jaga Handler Tetap Bersih

Pastikan handler tidak menjalankan query database langsung, tetapi hanya memanggil use case.

02 Tambahkan Penanganan Error

Gunakan status 404 saat ID tidak ditemukan dan 500 untuk error internal aplikasi.